

April 13, 2026

```
[105]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, getcontext # Besonders für sig.
↳ Runden
import math

# Berechnungen und Plotting
from scipy import signal
import scipy.optimize
import scipy.constants as c
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
# from scipy.signal import find_peaks, peak_widths
# from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial

%matplotlib widget
import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
```

```

import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Markdown, Math, Latex, HTML

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[106]: def test_stability(versuch):
        return Markdown(f"$\\rightarrow$ Instability of {versuch} confirmed")

```

Linear Stability Analysis

```

[107]: test_stability("Michelson-Interferometer")
        # evaluate Versuch 232

```

[107]: $\Rightarrow$ **Instability** of Michelson-Interferometer confirmed

```

[108]: currentversuch = "232"
        base = Path.cwd()
        Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
        ↪ # initializing paths
        Ausgabe_path = Path(base/"Diagramme")
        print(Messwerte_path)
        print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Messwerte\232

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Auswertungen\Code\232\Diagramme

Runden signifikanter Stellen

```

[109]: def round_to_sigs(val, errVal, einheiten):
        """
        Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
        ↪ Diese Funktion wurde etwas umständlicher
        geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
        ↪ Daher musste hier mit dezimal gearbeitet werden.

```

Zudem sollten besonders kleine und große Messwerte in der
 ↳ Dezimalschreibweise geschrieben werden, damit diese auch
 für Protokolle geeignet sind.

Parameter

****val**** : float

Messwert

****errVal**** : float

Ungenauigkeit des Messwertes

Return

****value_rounded**** : str

Gerundeter Messwert

****error_rounded**** : str

Gerundete Ungenauigkeit des Messwertes

****res**** : str

"Messwert \pm Fehler"

"""

Daten zu Dezimal wechseln, da Floats probleme machen

val = Decimal(str(val))

errVal = Decimal(str(errVal))

exp = int(math.floor(math.log10(float(errVal))))

Die erste

↳ signifikante Stellenposition wird anhand des errVal bestimmt

if round(float(errVal / (Decimal(10) ** exp))) < 3:

wenn 1,2,3

↳ erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition

↳ hinzu

exp -= 1

scale = Decimal(10) ** exp

val_round = (val / scale).quantize(Decimal('1'), rounding=ROUND_CEILING) *

↳ scale

mit scale wird das Komma so verschoben, dass alle

↳ signifikanten Stellen vor dem Komma stehen, und dann alle Nachkommastellen

↳ weggerundet werden

err_round = (errVal / scale).quantize(Decimal('1'), rounding=ROUND_CEILING)

↳ * scale

qty = f"\qty{{{val_round}}({err_round})}{{{einheiten}}}"

num = f"\num{{{val_round}}({err_round})}"

```

    return float(val_round), float(err_round), num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
    ↪excluded=['einheiten'])

    # wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
    ↪gewollt sind, die 10n/e Präfixe beinhalten)

```

Gauss'sche Fehlerfortpflanzung

```

[110]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
    ↪ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
    ↪genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    -----
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParameters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParameters = []

    function = sp.sympify(function)
    variables = errPronePar.split(",")
    variables = sp.symbols(variables)

    for variable in variables:
        Delta = sp.Symbol(f"Delta_{variable}")

```

```

        partial = sp.diff(function, variable)                                # Die Funktion
    ↪wird nach der fehlerbehafteten Variable abgeleitet
        term=sp.UnevaluatedExpr(partial*Delta)**2
        error = error + ((term))                                           # Fehler werden
    ↪quadratisch aufsummiert
        errProneParamaters.append((variable,Delta))

absolute_err=sp.simplify(sp.sqrt(error),rational = True)
relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)

if latex:
    #Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function,long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")
    display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
    print(sp.latex(absolute_err))
    print("Relativer Fehler:")
    display(Math(sp.latex(relative_err,long_frac_ratio=2)))
    print(sp.latex(relative_err))
    return function,absolute_err, relative_err, errProneParamaters

```

Sigma-Abweichungen

```

[111]: def sigma_abweichung(p1, p2, err_p1 = 0, err_p2 = 0):
    """
    Funktion zum berechnen der Sigma-Abweichung von zwei Messwerten, oder einem
    ↪Messwert und einem Literaturwert.
    """
    if err_p1 == 0 and err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
    ↪fehlerbehaftet sein!")
    else:
        abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

        exp = int(math.floor(math.log10(float(abweichung))))
        if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
    ↪1,2,3 erste signifikante Stelle, dann kommt eine zweite
    ↪Nachkommastellenposition hinzu
            exp -= 1
        scale = Decimal(10) ** exp
        abweichung_round = (abweichung / scale).quantize(Decimal('1'),
    ↪rounding=ROUND_CEILING) * scale
        res = f"\num{{{abweichung_round}}}"
        print(res)

```

```
[112]: x=np.array([2.951,2.954,2.970,2.966,2.829])*1e-3
Delta_x=np.sqrt(2)*9*1e-6
x,Delta_x,_,_=v_round(x,Delta_x,r"\m")
m=np.array([11194,11130,11177,11166,10613])
Lambda=2*x/m*1e9
Delta_lambda=2*sqrt(Delta_x**2/m**2)*1e9
Lambda,Delta_lambda,latexnum,_,_=v_round(Lambda,Delta_lambda,r"\nano\m")
for i in range(len(Lambda)):
    print(latexnum[i])
barlambda=np.mean(Lambda)
deltabarlambdastat=(np.std(Lambda)/5)
deltabarlambdasyst=np.mean(Delta_lambda)
barlambda,delta_barlambda,_,_=round_to_sigs(barlambda,deltabarlambdasyst+deltabarlambdastat,r"
print(barlambda)
sigma_abweichung(531.0,532.0,1,2.9)
```

\num{527.3(2.4)}

\num{530.9(2.4)}

\num{531.7(2.4)}

\num{531.5(2.4)}

\num{533.4(2.5)}

531.0

\num{0.4}

```
[135]: rings=np.arange(0,50,5)
p=[np.array([710,640,565,490,415,340,270,195,120,50]),np.
    ↪array([725,655,580,505,430,355,285,205,135,65]),np.
    ↪array([710,635,565,490,420,345,270,195,120,45])]
delta_p = np.sqrt(5**2 + (800*0.06)**2)

plt.close('all')
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
ax.set_title('Brechungsindex Luft')
ax.set_xlabel(r'durchlaufene Interferenzmaxima $m$ [count]')
ax.set_ylabel(r'Druck $p$ [Torr]')
# ax.set_ylim((3e1,1*1e2))
# ax.set_xlim(())
def lin(x,a,b):
    return a*x + b

for i in range(3):
    ax.errorbar(x=rings,y=p[i],yerr=delta_p,fmt=".",capsize=3,elinewidth=0.
    ↪5,label='')
    popt, pcov = curve_fit(lin, rings, p[i], sigma =_
    ↪delta_p,absolute_sigma=True)
```

```

    ax.plot(rings, lin(rings,*popt),r'-', color=f"C{i}",label='Linearer Fit')
    p_m, Delta_pm =round_to_sigs(popt[0], np.
    ↪sqrt(pcov[0][0]),einheiten=r"\Torr\per\count")[0],round_to_sigs(popt[0], np.
    ↪sqrt(pcov[0][0]),einheiten=r"\Torr\per\count")[1]
    print(p_m,Delta_pm)

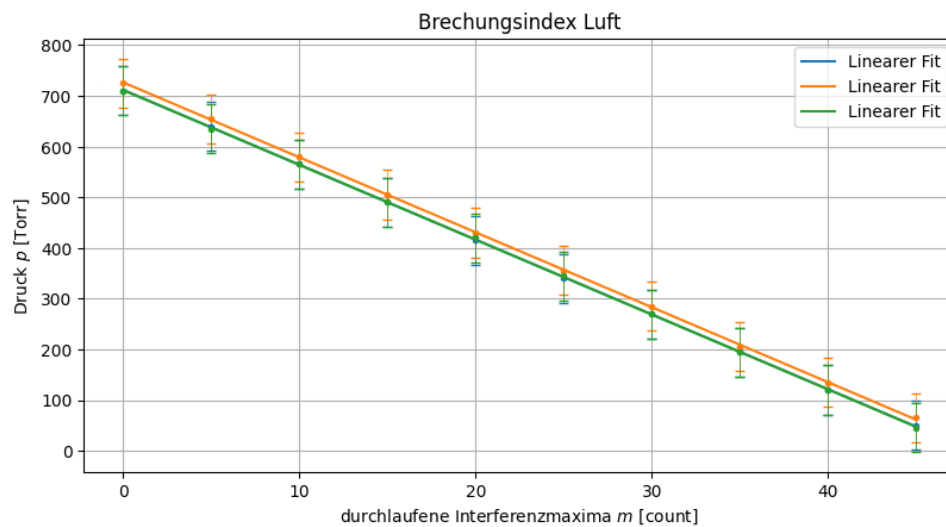
ax.grid()
ax.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.2_müberp.png",format="png")

```

```

-14.7 1.1
-14.7 1.1
-14.7 1.1

```



```

[114]: lamb=barlambd*1e-9
Delta_lamb=delta_barlambd*1e-9
a = 50e-3 #m
Delta_a = 0.05e-3 #m
T = 24.3 + 273.15 #K
Delta_T = 0.3#K
p_0 = 101325/133.3 #Torr Umrechnung
T_0 = 273.15#K
zeta=14.7
Delta_zeta=1.1

n_0 = (lamb * p_0 * T) / (2*a*T_0*zeta) + 1

```

```
Delta_n_0= (n_0-1) * np.sqrt((Delta_lamb/lamb)**2 + (Delta_a/a)**2 +(Delta_zeta/
↪zeta)**2 + (Delta_T/T)**2)
n_0,Delta_n_0,_,latexqty=round_to_sigs(n_0,Delta_n_0,r'')
print(latexqty)
sigma_abweichung(n_0,1.00028,Delta_n_0,0)
```

```
\qty{1.000300(0.000023)}{ }
\num{0.9}
```

```
[ ]: t,U=np.loadtxt(Messwerte_path/'TEK00008.CSV',delimiter="," ,skiprows=18,↪
↪unpack=True)
peakind = signal.find_peaks_cwt(U, np.arange(1,30),noise_perc=20)

plt.close('all')
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 13))

ax1.plot(t,U, linewidth=1,label='Oszilloskop Werte aus .csv Datei')
ax1.plot(t[peakind], U[peakind],color='red',marker='.',
↪',label='Peaks',linewidth=0)
ax1.set_xlabel('Zeit [s]')
ax1.set_ylabel('Intensitaet [b.E.]')
ax1.set_title('Interferogramm einer LED')
ax1.set_xlim([-0.05, 0.08]) # hier die Achsenbereiche

# Fitfunktion Gauss
x=np.linspace(-0.05,0.08,100)
def fitgauss(t, a, mu, sig):
    return a/np.sqrt(2*np.pi)/sig*np.exp(-(t-mu)**2/(2*sig**2))
popt, pcov= curve_fit(fitgauss, t[peakind], U[peakind])
ax2.plot(x, fitgauss(x, *popt),color='black',label='Fit')
ax2.plot(t[peakind], U[peakind],color='red',marker='.',
↪',label='Peaks',linewidth=0)
ax2.set_title('Gauß-Fit an Inteferogramm')
ax2.set_xlabel('Zeit [s]')
ax2.set_ylabel('Intensitaet [b.E.]')
ax2.set_xlim([-0.05, 0.08]) # hier die Achsenbereiche
ax2.text(0.04,0.3,r'\sigma=$'+f"{popt[2]:.5f}s")
ax1.legend(loc = 'best')
ax2.legend(loc = 'best')

sigma,Delta_sigma,_,latexsig=round_to_sigs(popt[2],np.sqrt(pcov[2][2]),r'\s')
FWHM=2*sigma*np.sqrt(2*np.log(2))
Delta_FWHM=2*Delta_sigma*np.sqrt(2*np.log(2))
FWHM,Delta_FWHM,_,latexfh=round_to_sigs(FWHM,Delta_FWHM,r'\s')
print(latexsig)
print(latexfh)
v=0.11*1e3 #micro m/s
```



```

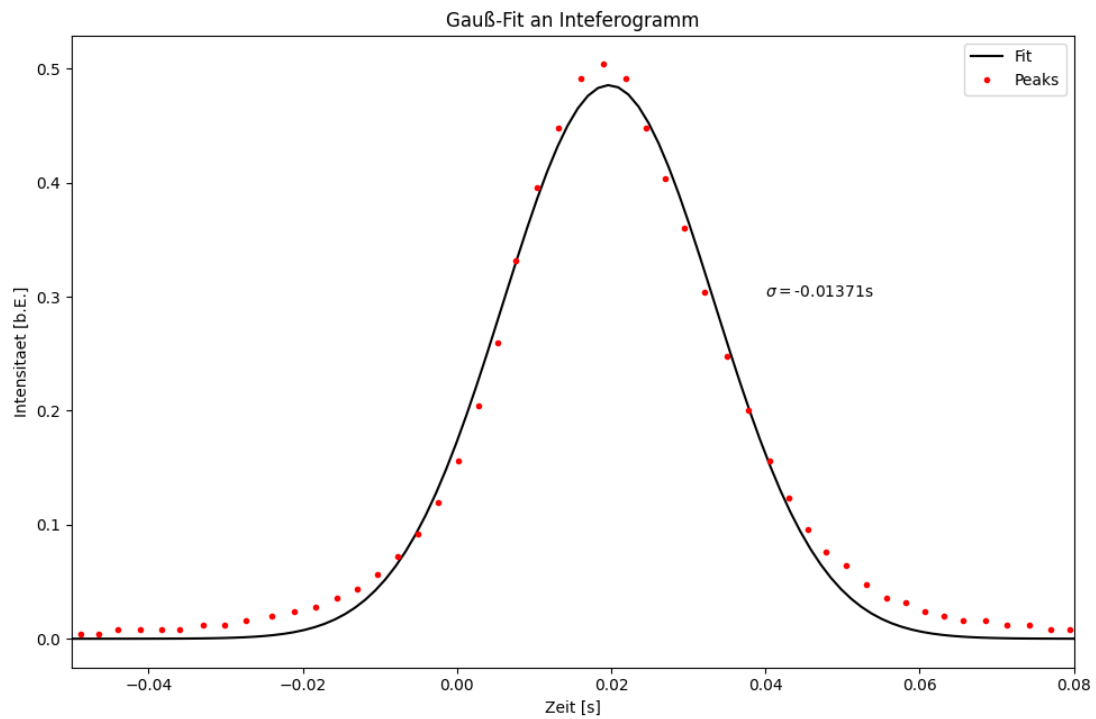
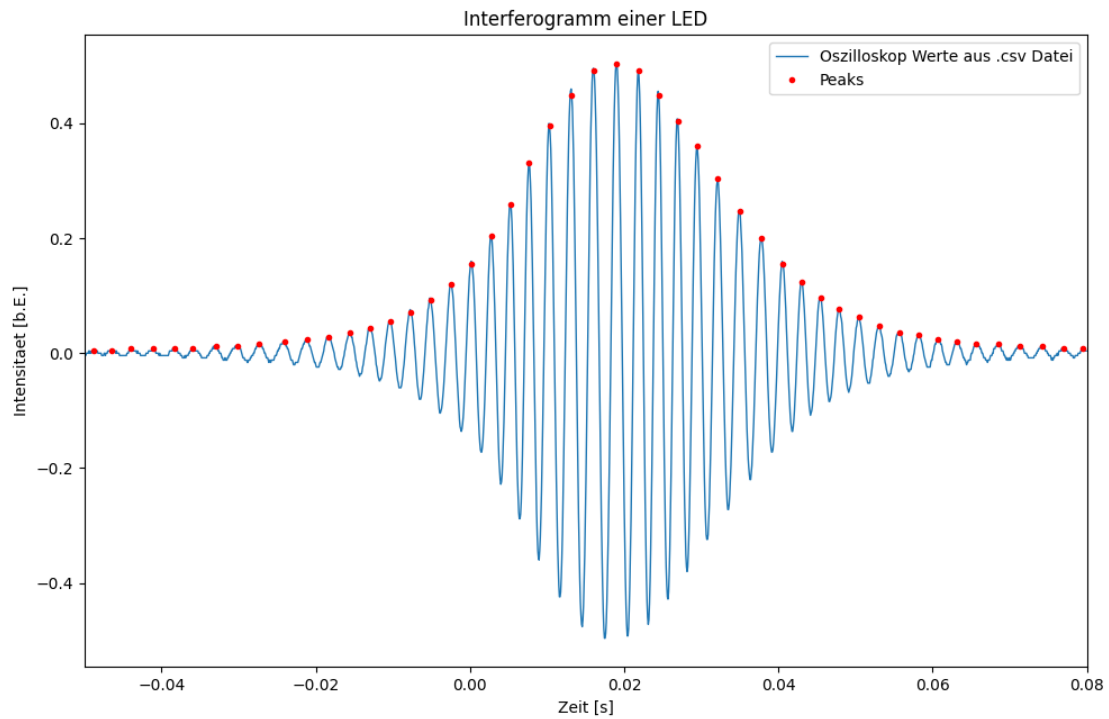
L_C=2*FWHM*v
Delta_L_C=2*v*Delta_FHWM
print(round_to_sigs(L_C,Delta_L_C,r'\micro\m')[3])
plt.tight_layout()
plt.show()
plt.savefig(Ausgabe_path/"Nr.3_kohärenz.png",format="png")

```

```

\qty{-0.01371(0.00019)}{\s}
\qty{-0.0322(0.0005)}{\s}
\qty{-7.08(0.11)}{\micro\m}

```



```
[121]: ggf("2*o*sqrt(2*ln(2))", "o")
```

Funktion:

$$2\sqrt{2}o\sqrt{\log(2)}$$

$$2 \sqrt{2} o \sqrt{\log(\left(2 \right))}$$

Absoluter Fehler:

$$2\sqrt{2}\sqrt{\Delta_o^2}\sqrt{\log(2)}$$

$$2 \sqrt{2} \sqrt{\Delta_o^2} \sqrt{\log(\left(2 \right))}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_o^2}{o^2}}$$

$$\sqrt{\frac{\Delta_o^2}{o^2}}$$

```
[121]: (2*sqrt(2)*o*sqrt(log(2)),
        2*sqrt(2)*sqrt(Delta_o**2)*sqrt(log(2)),
        sqrt(Delta_o**2/o**2),
        [(o, Delta_o)])
```